

CAPABILITIES, ISOLAMENTO E MENOR PRIVILÉGIO

*Por décadas o `ping` foi SUID: rodava **inteiro** como `root` — com direito a ler `/etc/shadow`, matar qualquer processo e montar sistemas de arquivos — só para mandar 64 bytes de ICMP. Hoje ele não tem SUID nenhum: tem `cap_net_raw`, e mais nada. **Menor privilégio deixou de ser conselho e virou engenharia.***

087
GUIAS LINUX

O PODER MONOLÍTICO DE ROOT, QUEBRADO EM PEÇAS NOMEADAS

UID 0
uma única verificação no kernel inteiro:
pode tudo
. Sem gradação entre "pode um pouco" e "pode tudo".

→

`CAP_NET_RAW`	socket cru e pacote forjado — **`ping`, `tcpdump`, `nmap -sS`**
`CAP_NET_BIND_SERVICE`	escutar em porta abaixo de 1024 **sem ser `root`**
`CAP_CHOWN`	trocar o dono de um arquivo — instaladores
`CAP_KILL`	sinalizar processo de outro dono — supervisores
`CAP_SYS_ADMIN`	montar, criar *namespace*, dezenas de outras — **equivale a root**

Cada peça é verificada de forma independente: o `ping` tem `CAP_NET_RAW` e nada mais, então uma falha nele rende forjar pacotes — não a máquina. **Mas a quebra não é uniforme:** `CAP_SYS_ADMIN` acumulou tantas operações que concedê-la é conceder `root` por um caminho mais longo. Capability só reduz privilégio **quando a peça é pequena**.

A LISTA QUE VALE LER ANTES DE CONCEDER QUALQUER COISA

PEÇAS PEQUENAS — REDUÇÃO REAL

fazem uma coisa, e a falha para nelas

`cap_net_raw`	socket cru: **`ping`, `dumpcap`, `mtr-packet`**
`cap_net_bind_service`	porta < 1024 sem root
`cap_net_admin`	interfaces, rotas, firewall (`ip`, `nft`, VPN)
`cap_chown`	dono de arquivo alheio — **com cautela: reescreve a rota até `root`**
`cap_kill`	sinalizar processo de outro usuário

PEÇAS QUE SÃO ROOT COM OUTRO NOME

conceder é entregar a máquina, mais devagar

`cap_sys_admin`	montar, *namespaces*, dezenas de operações
`cap_sys_module`	carregar módulo de kernel = **controle total**
`cap_dac_override`	ignora **toda** checagem de permissão: `/etc/shadow`, chaves privadas
`cap_setuid / setgid`	assumir outra identidade — escalada direta
`cap_sys_ptrace`	ler a memória de qualquer processo

NA PRÁTICA: LER, CONCEDER — E DEIXAR O SYSTEMD CONFINAR

kali@kali: ~

```
$ getcap /usr/bin/ping
/usr/bin/ping cap_net_raw=ep
$ sudo getcap -r / 2>/dev/null # o inventário inteiro
# lista curta e explicável, ou há pergunta a fazer
$ capsh --print | head -3        # do processo atual
$ capsh --decode=0000000000003000 # hex → nomes
$ sudo setcap cap_net_bind_service=+ep ./meuservidor
# e/p/i = efetivo (ativo agora) / permitido (o teto) /
# herdável. O conjunto LIMITANTE é o teto absoluto:
$ sudo capsh --drop=cap_net_raw -- -c 'ping -c1 1.1.1.1'
ping: socktype: SOCK_RAW  (falha até sob sudo)
# no_new_privs: nem eu nem meus filhos ganham privilégio
$ setpriv --no-new-privs sudo id
sudo: efetivo uid não é 0 — a escalada por SUID acabou
$ systemd-analyze security          # placar geral
$ systemd-analyze security meuservico.service
```

O SANDBOX QUE CABE NUM ARQUIVO DE UNIDADE

```
[Service]
User=meuservico
NoNewPrivileges=yes            # fecha o SUID
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_BIND_SERVICE
ProtectSystem=strict           # tudo só-leitura
ProtectHome=yes                # /home e /root somem
PrivateTmp=yes                 # /tmp só dele
StateDirectory=meuservico      # o único gravável
ProtectKernelTunables=yes      # /proc/sys travado
ProtectKernelModules=yes       # não carrega módulo
PrivateDevices=yes             # só /dev/null e cia.
ProtectProc=invisible          # não vê outro PID
RestrictNamespaces=yes
MemoryDenyWriteExecute=yes
SystemCallFilter=@system-service
```

Por baixo são *namespaces*, cgroups e seccomp — os mecanismos do contêiner, **em forma declarativa e sem tocar no código. Aperte o parafuso aos poucos, com `journalctl -fu` aberto:** essas diretivas quebram serviço de forma opaca.

Nunca conceda capability (nem SUID, nem regra de `sudo`) a um interpretador. Um `setcap cap_setuid=+ep /usr/bin/python3` transforma o Python numa máquina de escalada de duas linhas: `import os; os.setuid(0); os.system("/bin/bash")` — e qualquer usuário vira `root`. Vale para `perl`, `node`, `gdb` e todo programa com escape para shell: `NOPASSWD: /usr/bin/find` parece restrito e não é, porque `find . -exec /bin/sh \;` é um shell como `root`. **Privilégio dado a quem executa código arbitrário é privilégio irrestrito** — metade da escalada local que existe cabe nessa frase.

RESUMO RÁPIDO

- ✓ **O kernel quebrou `root` em ~40 peças verificadas independentemente.** O `ping` moderno tem `cap_net_raw` e mais nada.
- ✓ **A quebra não é uniforme:** `cap_sys_admin`, `cap_sys_module`, `cap_dac_override` e `cap_setuid` são `root` por caminho mais longo.
- ✓ **Capability vai em binário que faz uma coisa só**, com entrada controlada — nunca em interpretador.
- ✓ **`no_new_privs` apaga a escalada por SUID** em toda a árvore de processos, e é de mão única. É a base de todo sandbox moderno.
- ✓ **Copiar um arquivo não copia sua capability** — nem o bit SUID. Bom lembrete ao empacotar ou restaurar backup.
- ✓ **`systemd-analyze security` dá nota a cada serviço** e converte "deveríamos endurecer isso" em lista priorizada.

NO KALI

O Kali é um museu vivo de capability aplicada certo, porque as ferramentas de rede precisam de privilégio e não podem ser SUID: `sudo getcap -r /usr` mostra o `dumpcap` com `cap_net_raw,cap_net_admin=eip` — a peça que deixa o Wireshark capturar com o usuário `kali`. `sudo dpkg-reconfigure wireshark-common` **é a decisão certa** para capturar sem `sudo`: cria o grupo `wireshark`, dá a capability só ao `dumpcap` e restringe a execução ao grupo. Rodar o Wireshark inteiro como `root` exporia um decodificador gigantesco — generoso em CVEs — a tráfego arbitrário, com privilégio total. É o exemplo didático perfeito: privilégio mínimo, no menor componente, pelo menor tempo. **O `nmap` moderno não é SUID, e isso muda o resultado:** `nmap -sS` precisa de socket cru e, sem `CAP_NET_RAW` ou `sudo`, cai **em silêncio** para `-sT` (`connect()`) — mais lento, mais barulhento no log do alvo e com resultado diferente. Ver o `nmap` "funcionando" e devolvendo coisa estranha quase sempre é isto. **Enumerar capability e SUID é etapa padrão de pós-exploração — por isso enumere primeiro na sua máquina:** `sudo getcap -r /` e `sudo find / -perm -4000 -type f` levam dez segundos e respondem a pergunta que importa; é o que o `linpeas` faz. Contra sistema de terceiro sem autorização escrita, é crime pela Lei 12.737/2012. **`systemd-analyze security` na sua VM é uma revelação desconfortável** — vários serviços comuns passam de 8. Num laboratório nem todos precisam ser corrigidos, mas o exercício ensina a ler o placar, e o mesmo comando em produção vira lista de tarefas. **RHEL:** capabilities e diretivas do systemd são idênticas (são do kernel), mas o SELinux empilha por cima — um `httpd_t` com todas as capabilities do mundo continua sem ler `/home` se a política não permitir. Duas verificações desacopladas: o invasor precisa das duas.